Laboratorio di Programmazione

Corso di Laurea in Informatica

Gr. 3 (N-Z)

Università degli Studi di Napoli Federico II

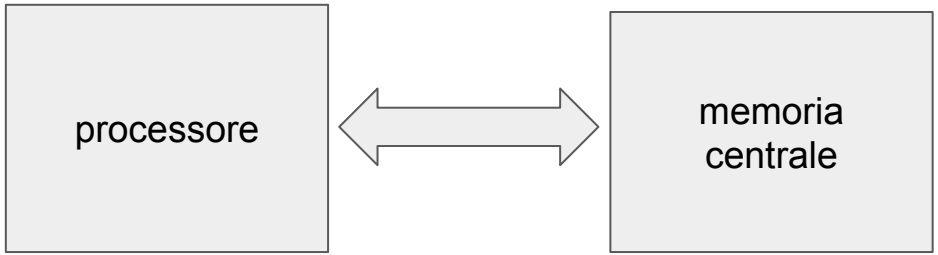
A.A. 2022/23

A. Apicella

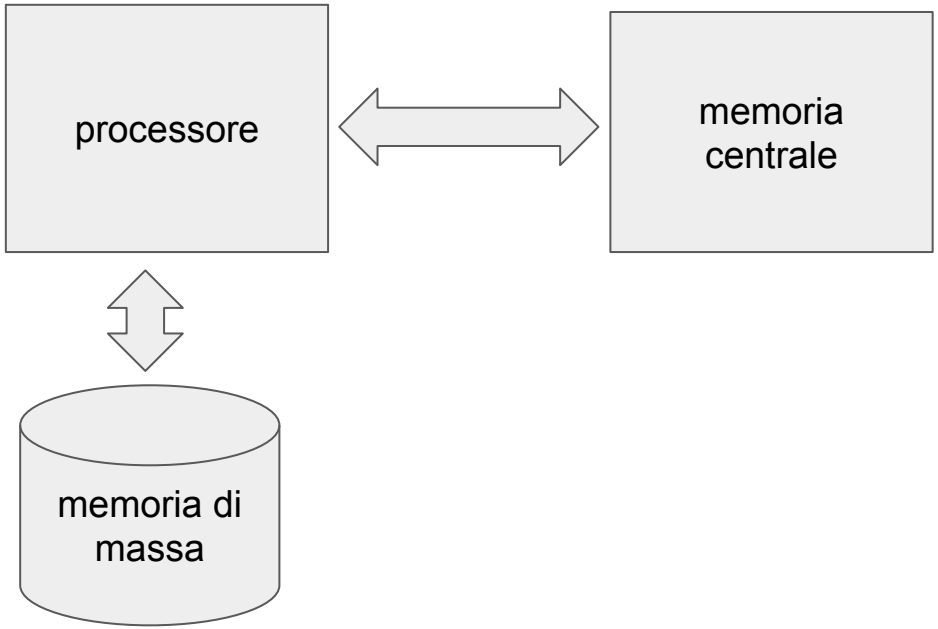
I/O su file

- Può capitare che un programma abbia necessità di leggere dati non da stdin (i.e. la tastiera), ma da file esterni. E.g., quando caricate il “salvataggio” di una partita ad un videogioco
- Quando un programma termina, dato che tutti i suoi dati sono in memoria centrale, questi vengono persi. Può essere necessario memorizzare dei dati su memoria di massa. E.g., quando salvate i dati di una partita ad un videogioco

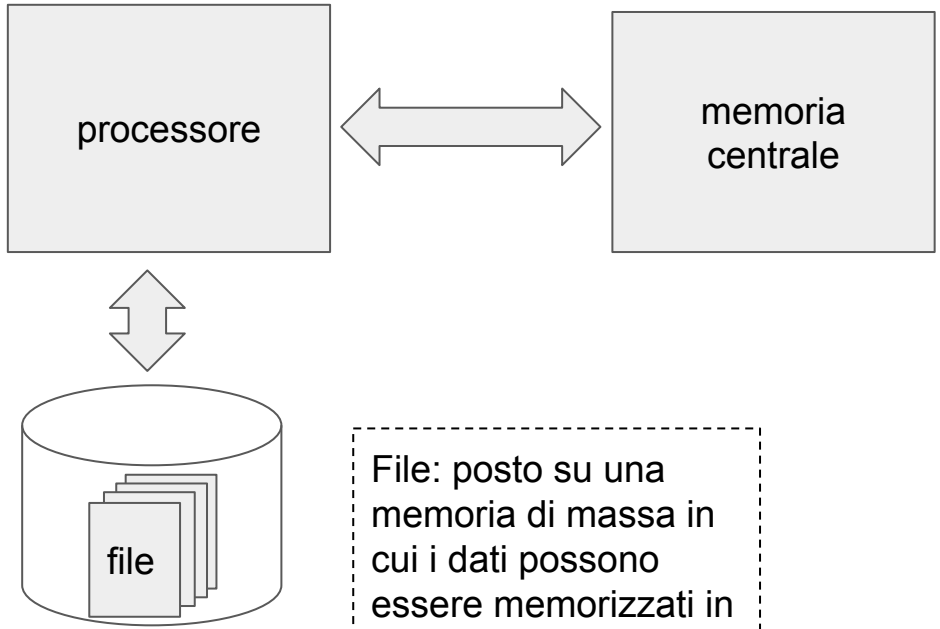
I/O su file



I/O su file



I/O su file



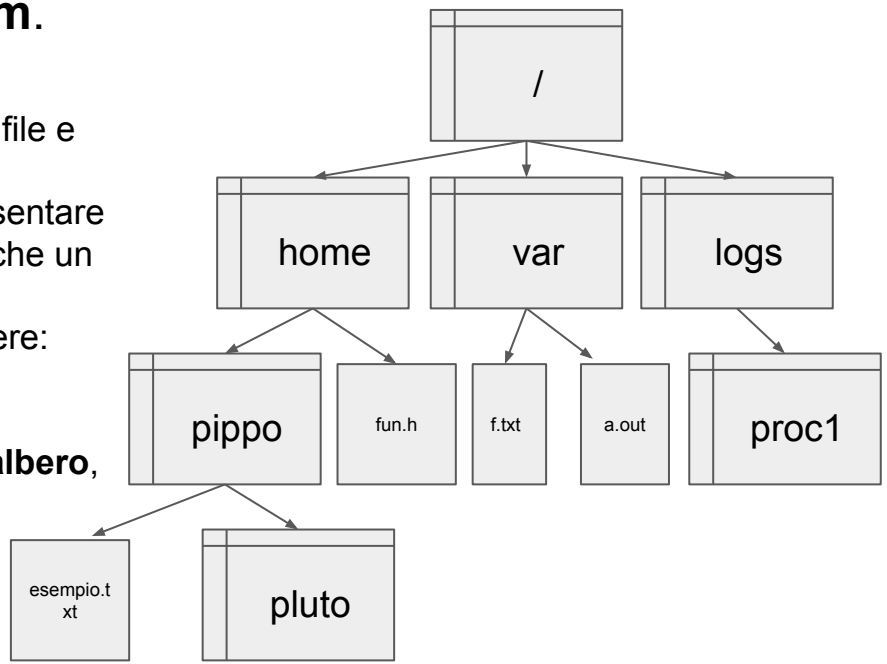
File: posto su una memoria di massa in cui i dati possono essere memorizzati in maniera non volatile

File

L’insieme delle informazioni memorizzate sulla memoria di massa è organizzato secondo una struttura che detta **filesystem**.

- I sistemi Linux e Windows utilizzano i concetti comuni di file e directory (anche dette “cartelle” in Windows).
- Un file è un insieme di informazioni che possono rappresentare dei dati (un testo, un’immagine, una canzone, etc.) o anche un programma (ad esempio un file eseguibile)
- Una directory è una sorta di contenitore che può contenere:
  - file
  - altre directory

  si viene quindi a delineare una struttura **gerarchica ad albero**, detto *directory tree* o *albero delle directory*.



File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (cartelle, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

```
/home/pippo/esempio.txt
```

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (cartelle, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

`/home/pippo/esempio.txt`

estensione

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (cartelle, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

`/home/pippo/esempio.txt`

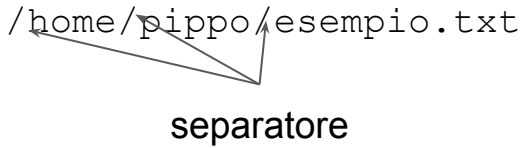
nome

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (cartelle, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:



File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (cartelle, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

`/home/pippo/esempio.txt`

prima directory da attraversare

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (*cartelle*, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

`/home/pippo/esempio.txt`

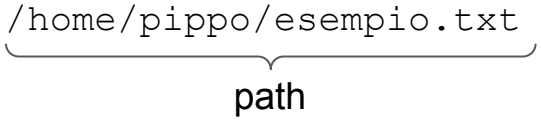
seconda directory
da attraversare

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path è composto da:

- la sequenza delle *directory* (*cartelle*, in Windows) da “attraversare” per trovare il file
  - ogni directory è separata da un carattere separatore (“`/`” nei sistemi Linux, “`\`” nei sistemi windows)
- il nome del file
- l’estensione estensione
  - preceduta da un “.”
  - generalmente (ma non più obbligatoriamente) di 3 caratteri
  - nei sistemi Windows, determina il tipo di file. Nei sistemi Linux la sua presenza non è obbligatoria, in quanto il tipo è ricavato in altro modo

Esempio:

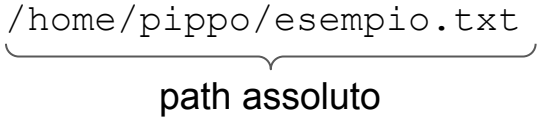


File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path può essere:

- **assoluto:** comprende l'intera sequenza di directory da attraversare
  - su sistemi Linux, inizia sempre con "`/`"

Esempio:



File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path può essere:

- **assoluto:** comprende l'intera sequenza di directory da attraversare
  - su sistemi Linux, inizia sempre con "`/`"
- **relativo:** comprende le directory da attraversare *da un dato punto in poi*
  - su sistemi Linux, inizia con "`./`"

Esempio:

`./pippo/esempio.txt`

path relativo a partire dalla directory `/home`

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path può essere:

- **assoluto:** comprende l'intera sequenza di directory da attraversare
  - su sistemi Linux, inizia sempre con "`/`"
- **relativo:** comprende le directory da attraversare *da un dato punto in poi*
  - su sistemi Linux, inizia con "`./`"

Esempio:

`./esempio.txt`

path relativo a partire dalla directory `/home/pippo`

File

I sistemi operativi con file system ad albero identificano un dato file attraverso un percorso (*path*). Un path può essere:

- **assoluto:** comprende l'intera sequenza di directory da attraversare
  - su sistemi Linux, inizia sempre con "`/`"
- **relativo:** comprende le directory da attraversare *da un dato punto in poi*
  - su sistemi Linux, inizia con "`./`" (in certi casi può essere omesso)
  - E' possibile utilizzare un path relativo a partire dalla directory in cui ci si trova
  - Per conoscere la directory in cui ci si trova, Linux mette a disposizione il comando `pwd` *(Print Working Directory)*
  - Si ricorda che, in Linux, è possibile muoversi attraverso le directory utilizzando il comando `cd` (*Change Directory*)
  - può contenere "`../`" per indicare che è necessario "tornare indietro" di una directory.

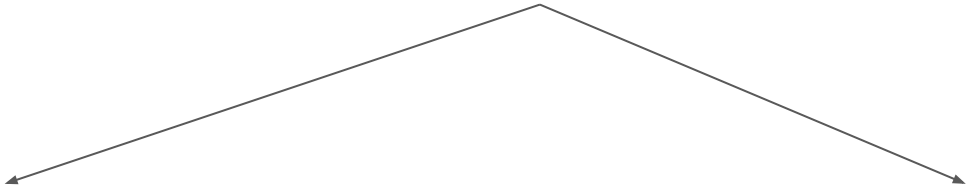
Esempio: ci si trova nella directory `/home/pippo, e` si vuole accedere al file `f.h` contenuto nella directory `/home/` è possibile accedervi tramite:

- path assoluto: `/home/f.h`
- path relativo: `./../f.h` oppure `../f.h` oppure `../../home/f.h` oppure `../pippo/../f.h`

I/O su file

Tipi di file

Il C è in grado di gestire 2 tipi di file:



file **binari**

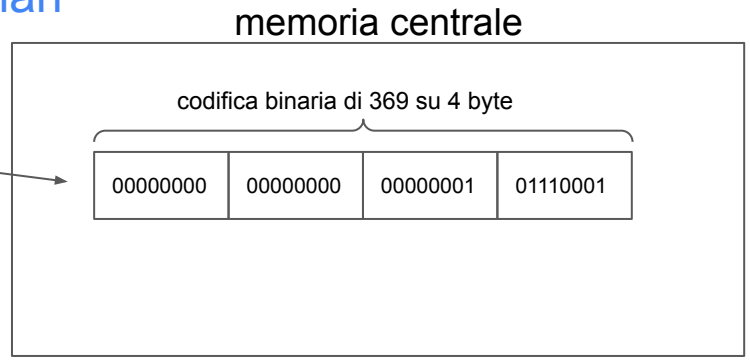
file **di testo** (o ASCII)

generalmente con estensione *.txt*

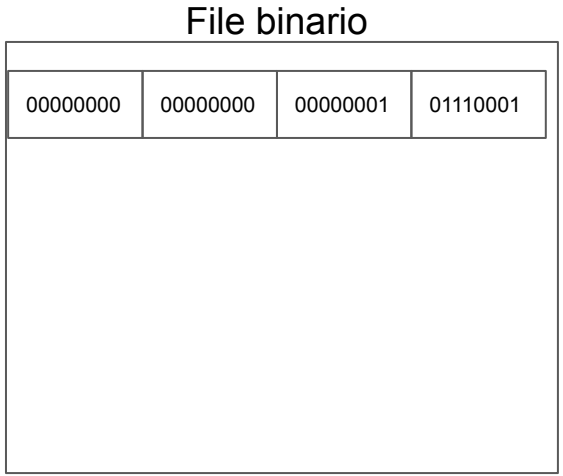
I/O su file

File binari

```
int a = 369;
```



il contenuto del file rispecchia fedelmente il contenuto della memoria centrale



I/O su file

File di testo

```
int a = 369;
```

memoria centrale

codifica binaria di 369 su 4 byte

00000000	00000000	00000001	01110001

il contenuto del file è la codifica ASCII di quanto contenuto in memoria centrale

tabella ASCII

Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char
00000000	0	Null	00100000	32	Spc	01000000	64	@	01100000	96	`
00000001	1	Start of heading	00100001	33	!	01000001	65	A	01100001	97	a
00000010	2	Start of text	00100010	34	"	01000010	66	B	01100010	98	b
00000011	3	End of text	00100011	35	#	01000011	67	C	01100011	99	c
00000100	4	End of transmit	00100100	36	$	01000100	68	D	01100100	100	d
00000101	5	Enquiry	00100101	37	%	01000101	69	E	01100101	101	e
00000110	6	Acknowledge	00100110	38	&	01000110	70	F	01100110	102	f
00000111	7	Audible bell	00100111	39	'	01000111	71	G	01100111	103	g
00001000	8	Backspace	00101000	40	(	01001000	72	H	01101000	104	h
00001001	9	Horizontal tab	00101001	41	)	01001001	73	I	01101001	105	i
00001010	10	Line feed	00101010	42	*	01001010	74	J	01101010	106	j
00001011	11	Vertical tab	00101011	43	+	01001011	75	K	01101011	107	k
00001100	12	Form Feed	00101100	44	,	01001100	76	L	01101100	108	l
00001101	13	Carriage return	00101101	45	-	01001101	77	M	01101101	109	m
00001110	14	Shift out	00101110	46	.	01001110	78	N	01101110	110	n
00001111	15	Shift in	00101111	47	/	01001111	79	O	01101111	111	o
00010000	16	Data link escape	00110000	48	0	01010000	80	P	01110000	112	p
00010001	17	Device control 1	00110001	49	1	01010001	81	Q	01110001	113	q
00010010	18	Device control 2	00110010	50	2	01010010	82	R	01110010	114	r
00010011	19	Device control 3	00110011	51	3	01010011	83	S	01110011	115	s
00010100	20	Device control 4	00110100	52	4	01010100	84	T	01110100	116	t
00010101	21	Neg. acknowledge	00110101	53	5	01010101	85	U	01110101	117	u
00010110	22	Synchronous idle	00110110	54	6	01010110	86	V	01110110	118	v
00010111	23	End trans. block	00110111	55	7	01010111	87	W	01110111	119	w
00011000	24	Cancel	00111000	56	8	01011000	88	X	01111000	120	x
00011001	25	End of medium	00111001	57	9	01011001	89	Y	01111001	121	y
00011010	26	Substitution	00111010	58	:	01011010	90	Z	01111010	122	z
00011011	27	Escape	00111011	59	;	01011011	91	[	01111011	123	{
00011100	28	File separator	00111100	60	<	01011100	92	\	01111100	124	|
00011101	29	Group separator	00111101	61	=	01011101	93	]	01111101	125	}
00011110	30	Record Separator	00111110	62	>	01011110	94	^	01111110	126	~
00011111	31	Unit separator	00111111	63	?	01011111	95	_	01111111	127	Del

codifica ASCII del carattere 3

File di testo

00110011	00110110	00111001

codifica ASCII del carattere 9

codifica ASCII del carattere 6

I/O su file

File di testo

```
int a = 369;
```

memoria centrale

codifica binaria di 369 su 4 byte

00000000	00000000	00000001	01110001

il contenuto del file è la codifica ASCII di quanto contenuto in memoria centrale

tabella ASCII

Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char
00000000	0	Null	00100000	32	Spc	01000000	64	@	01100000	96	`
00000001	1	Start of heading	00100001	33	!	01000001	65	A	01100001	97	a
00000010	2	Start of text	00100010	34	"	01000010	66	B	01100010	98	b
00000011	3	End of text	00100011	35	#	01000011	67	C	01100011	99	c
00000100	4	End of transmit	00100100	36	$	01000100	68	D	01100100	100	d
00000101	5	Enquiry	00100101	37	%	01000101	69	E	01100101	101	e
00000110	6	Acknowledge	00100110	38	&	01000110	70	F	01100110	102	f
00000111	7	Audible bell	00100111	39	'	01000111	71	G	01100111	103	g
00001000	8	Backspace	00101000	40	(	01001000	72	H	01101000	104	h
00001001	9	Horizontal tab	00101001	41	)	01001001	73	I	01101001	105	i
00001010	10	Line feed	00101010	42	*	01001010	74	J	01101010	106	j
00001011	11	Vertical tab	00101011	43	+	01001011	75	K	01101011	107	k
00001100	12	Form Feed	00101100	44	,	01001100	76	L	01101100	108	l
00001101	13	Carriage return	00101101	45	-	01001101	77	M	01101101	109	m
00001110	14	Shift out	00101110	46	.	01001110	78	N	01101110	110	n
00001111	15	Shift in	00101111	47	/	01001111	79	O	01101111	111	o
00010000	16	Data link escape	00110000	48	0	01010000	80	P	01110000	112	p
00010001	17	Device control 1	00110001	49	1	01010001	81	Q	01110001	113	q
00010010	18	Device control 2	00110010	50	2	01010010	82	R	01110010	114	r
00010011	19	Device control 3	00110011	51	3	01010011	83	S	01110011	115	s
00010100	20	Device control 4	00110100	52	4	01010100	84	T	01110100	116	t
00010101	21	Neg. acknowledge	00110101	53	5	01010101	85	U	01110101	117	u
00010110	22	Synchronous idle	00110110	54	6	01010110	86	V	01110110	118	v
00010111	23	End trans. block	00110111	55	7	01010111	87	W	01110111	119	w
00011000	24	Cancel	00111000	56	8	01011000	88	X	01111000	120	x
00011001	25	End of medium	00111001	57	9	01011001	89	Y	01111001	121	y
00011010	26	Substitution	00111010	58	:	01011010	90	Z	01111010	122	z
00011011	27	Escape	00111011	59	;	01011011	91	[	01111011	123	{
00011100	28	File separator	00111100	60	<	01011100	92	\	01111100	124	|
00011101	29	Group separator	00111101	61	=	01011101	93	]	01111101	125	}
00011110	30	Record Separator	00111110	62	>	01011110	94	^	01111110	126	~
00011111	31	Unit separator	00111111	63	?	01011111	95	_	01111111	127	Del

codifica ASCII del carattere 3

codifica ASCII del carattere 6

File di testo

00110011	00110110	00111001

codifica ASCII del carattere 9

visualizzabile con un semplice editor di testo